

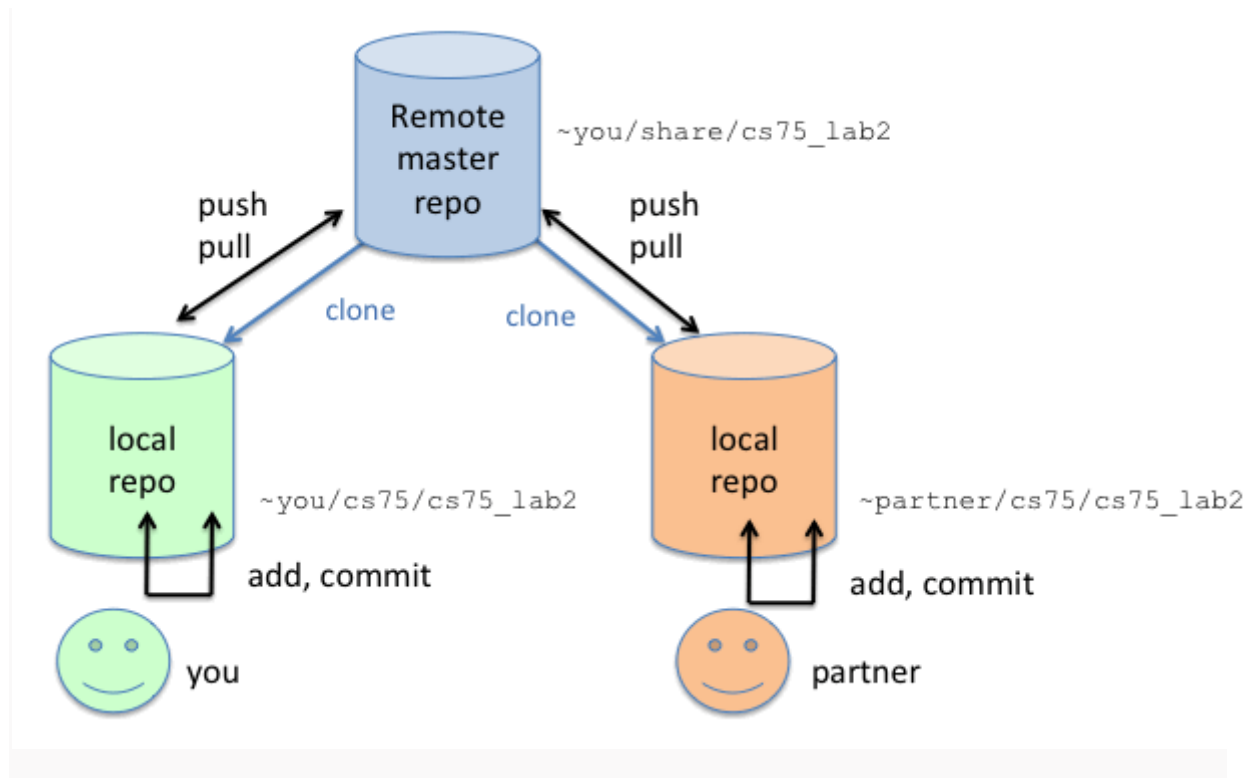
## Travaux pratiques 2 : git utilisation en groupe

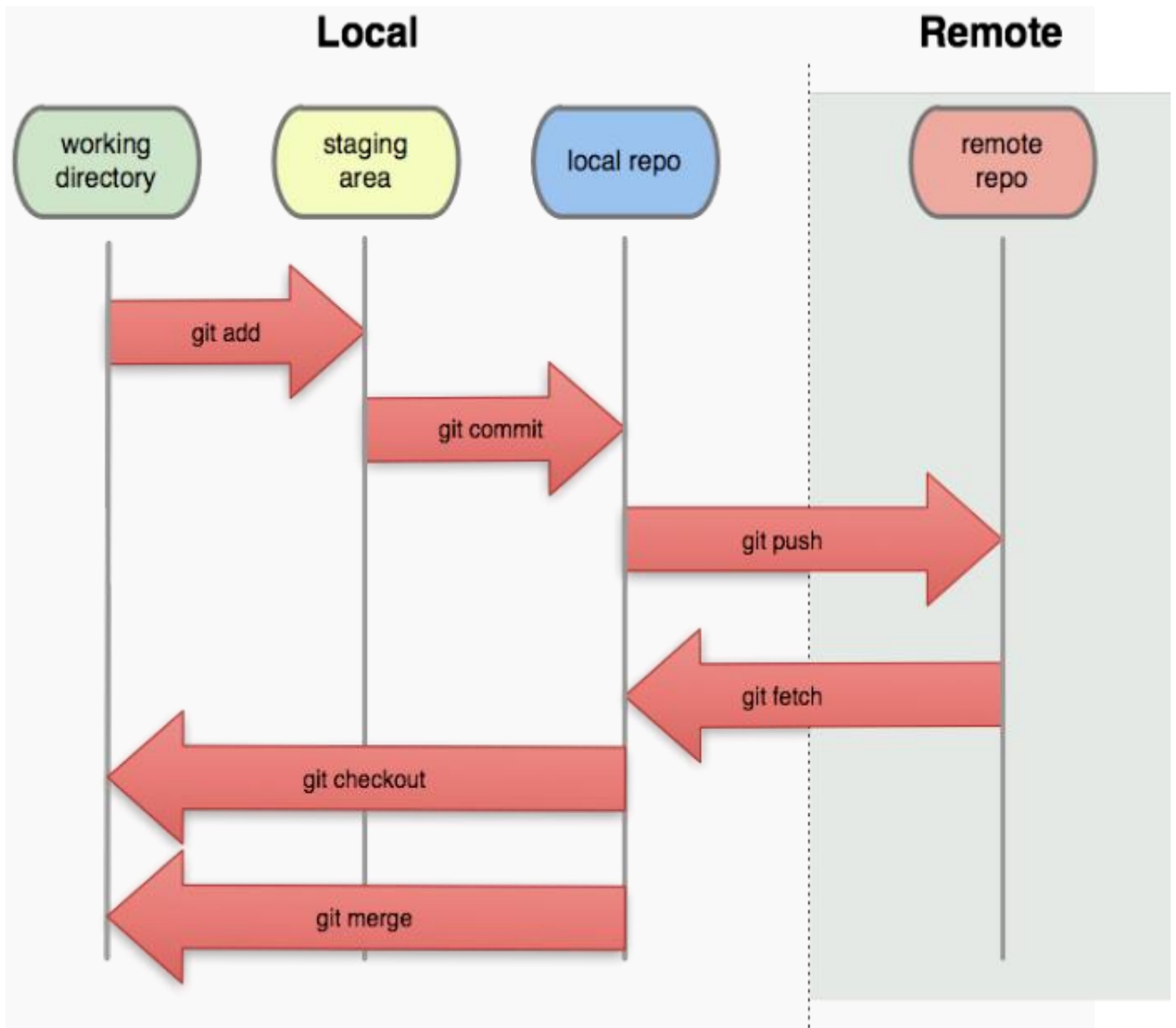
### Rappel TP1

Les commandes utilisées au TP précédent

- `git init`
- `git status`
- `git log`
- `git add`
- `git commit`
- `git reset`
- `git revert`
- `git checkout`

### Le dépôt distant





## Le "pull request" (ou "merge request")

### Contexte

Le code sur lequel vous allez travailler est un petit site web (<https://choosealicense.com>), dont le code est disponible sur <https://github.com/github/choosealicense.com>. Le code a été copié sur une machine sur le réseau dont l'adresse IP vous sera donnée.

Le projet git est hébergé sur le programme gitlab, il faut en premier vous créer un compte. Il suffit d'accéder à l'adresse IP donnée lors du TP, puis se créer un compte.

### Exercice 1 : récupérer le projet git

Faire un clone du dépôt git distant `http://IP/root/choosealicense.git` (l'adresse IP vous sera donnée lors du TP). Cela va créer un nouveau dossier "choosealicense"

Cette commande est l'équivalent du git init du TP précédant

```
git clone http://10.0.81.12/root/choosealicense
cd choosealicense
```

Maintenant, vous avez un "remote" qui s'appelle "origin", c'est le dépôt distant

```
git remote
# origin
```

Vous êtes sur la branche "gh-pages", et le remote a aussi une branche "gh-pages", qui s'appelle "origin/gh-pages" ou "remotes/origin/gh-pages". Vous pouvez récupérer la liste des branches distantes avec `-r` (pour "remote") ou `-a` (pour "all", soit locales et remote).

```
git branch
# * gh-pages

git branch -a
# * gh-pages
# remotes/origin/gh-pages
```

Votre branche "gh-pages" est par défaut une branche de suivie. Vous pouvez faire `git status` pour vous en assurer (il faut voir `Your branch is up-to-date with 'origin/gh-pages'.`).

```
git status
# On branch gh-pages
# Your branch is up-to-date with 'origin/gh-pages'.
# nothing to commit, working tree clean
```

Quand vous faites

- `git pull` (tirer en anglais, pour récupérer de la donnée), vous mergez la branche "origin/gh-pages" dans votre branche locale "gh-pages"
- `git push` (pousser en anglais, pour envoyer de la donnée), vous mettez à jour sur le dépôt distant le pointeur de branche "origin/gh-pages" sur votre dernier commit
- `git fetch`, vous permet de récupérer le contenu du dépôt distant, mais sans l'appliquer en local

Attention : pour faire git push vous devez être à jour de la branche "origin/gh-pages" (c'est-à-dire faire git pull avant)

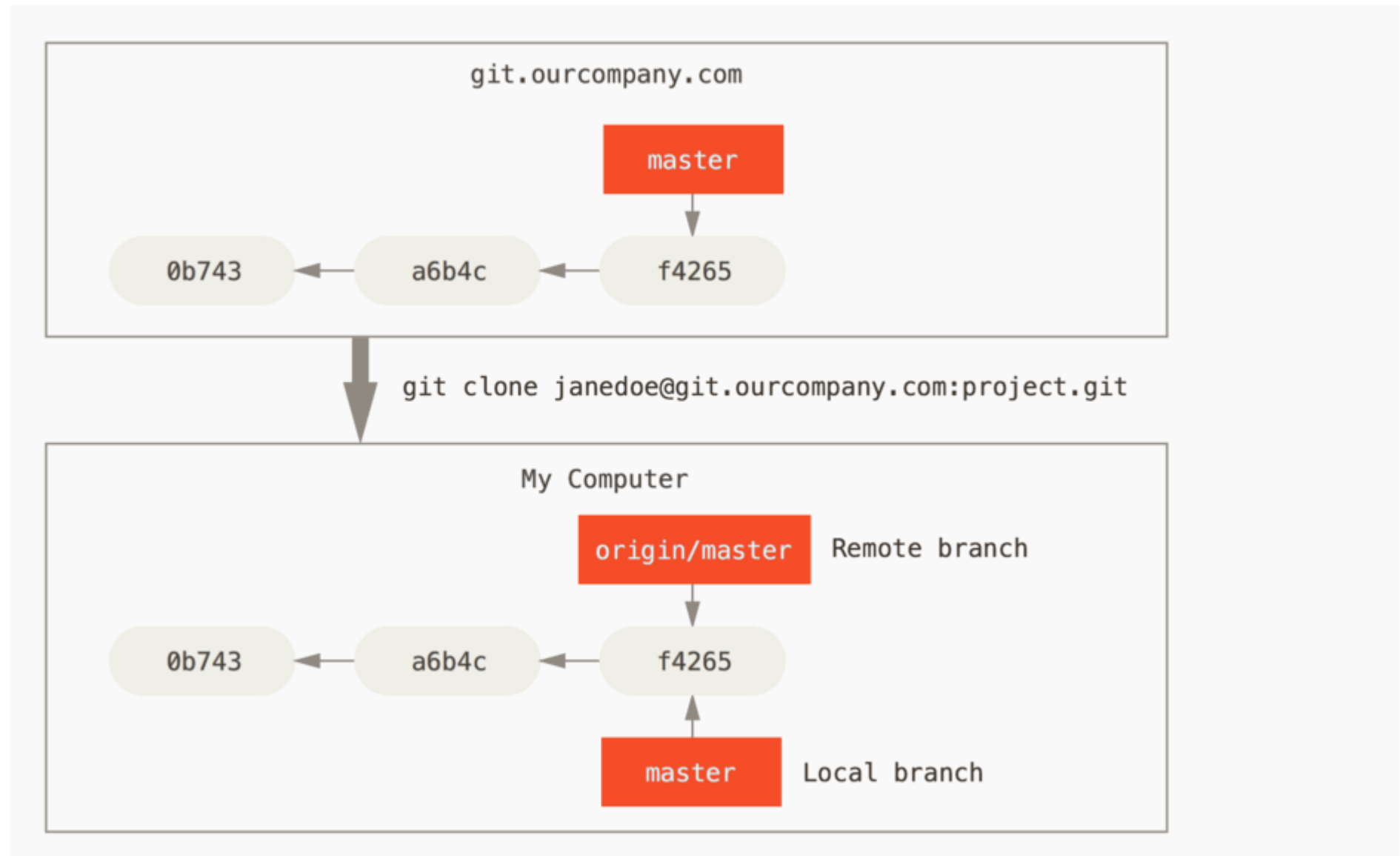
## Exercice 2 : revue de l'historique

- Quel est le numéro de la dernière pull request ?
- Combien de commit contient la pull request #550 ?
- Quel est le numéro, combien de commit, et quel est l'auteur dans la pull request du commit de merge 8061f2f ?

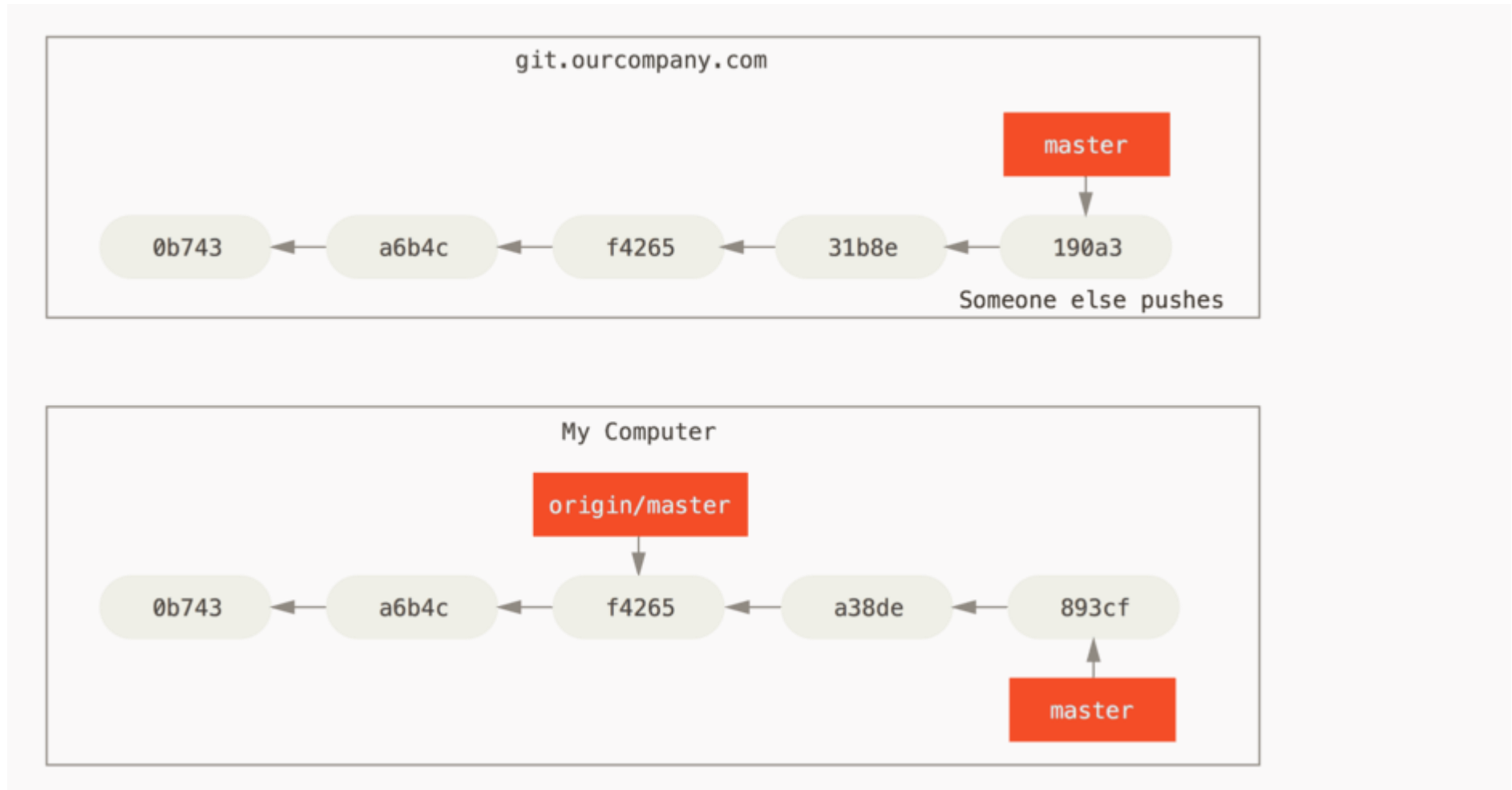
## Exercice 3 : récupération de versions ultérieures

- Quelle est la commande pour revenir à la version précédente sur le fichier "index.html" ?
- Un plugin javascript a été supprimé dans le commit b30306e502b99aabab256a5d2f68e8d50ba5072a, quel est le nom du plugin supprimé ? Quelle est la commande pour voir les fichiers modifiés dans ce commit ?
- Quelle est la commande pour récupérer le plugin supprimé ?

## Le pull / push

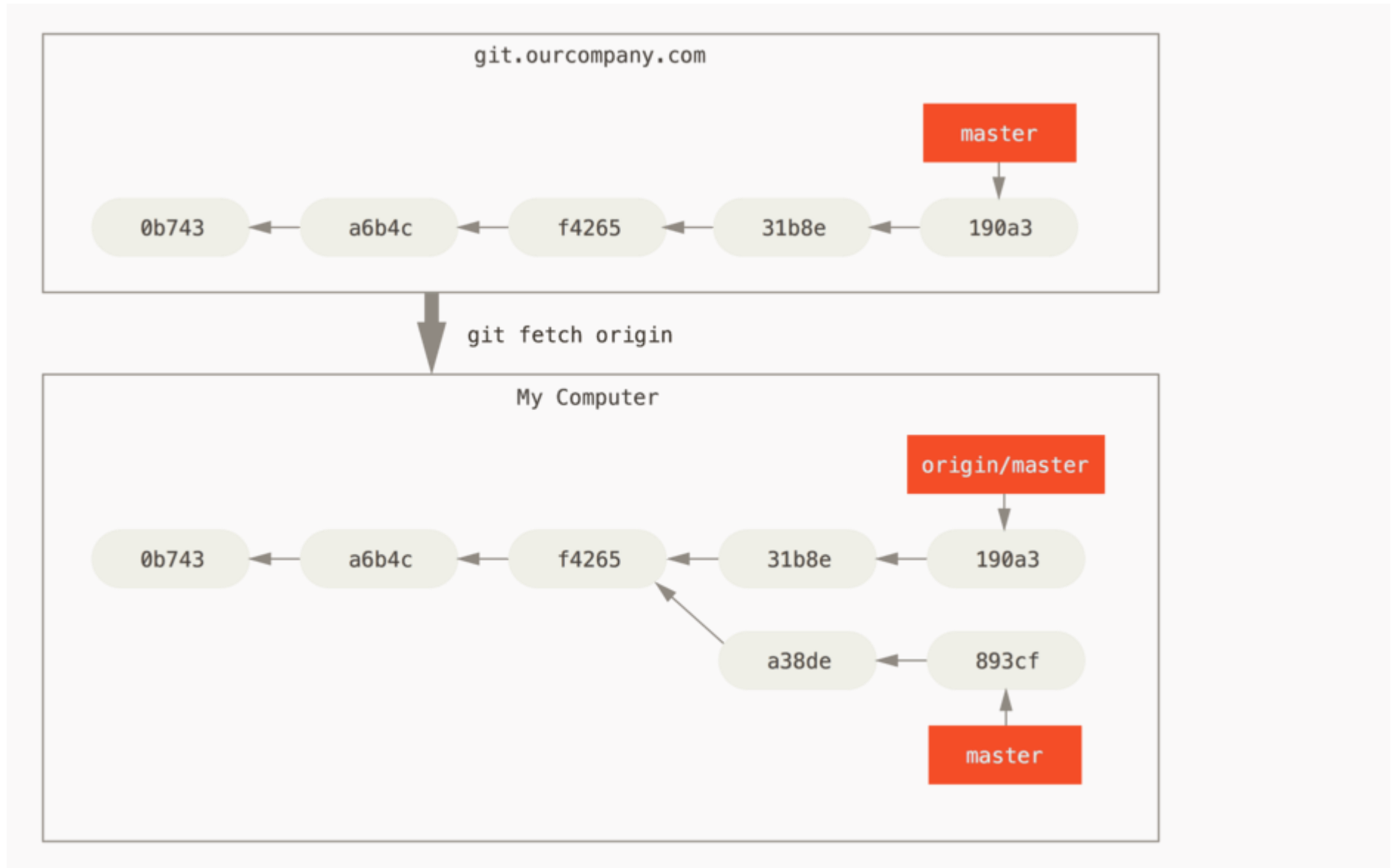












## Exercice 4 : envoyer un commit sur le dépôt distant

Le dossier "\_licenses" contient les licences qui sont affichées sur le site. Vous allez ajouter une license à votre nom (par exemple "alexandre-dubreuil-1.0.txt") et commiter le résultat.

Pour savoir si vous êtes à jour avec le dépôt distant, vous pouvez faire un "git status".

```
git status
# On branch gh-pages
# Your branch is ahead of 'origin/gh-pages' by 1 commit.
#   (use "git push" to publish your local commits)
# nothing to commit, working tree clean
```

Dans ce cas, vous avez un commit en plus en local qui n'est pas sur le distant (Your branch is ahead of 'origin/gh-pages' by 1 commit.). Pour envoyer votre dernier commit, il faut faire un push.

```
git push
# Username for 'http://IP': username
# Password for 'http://username@IP':
# Counting objects: 3, done.
# Delta compression using up to 8 threads.
# Compressing objects: 100% (3/3), done.
# Writing objects: 100% (3/3), 332 bytes | 0 bytes/s, done.
# Total 3 (delta 2), reused 0 (delta 0)
# To http://IP/root/choosealicense.git
#    9d9fe7d..93848fa  gh-pages -> gh-pages
```

On voit le gh-pages qui a avancé son pointeur de branche, 9d9fe7d..93848fagh-pages -> gh-pages, donc le dépôt distant pointe maintenant sur votre dernier commit.

**Si vous n'êtes pas le premier à pusher, git va vous demander de faire un git pull avant de pouvoir faire un git push. Lorsque vous faites un git push vous devez être à jour avec le distant, c'est-à-dire que vous devez avoir tous les commits du distant en local.**

```
git push
# Username for 'http://IP': username
# Password for 'http://username@IP':
# To http://IP/root/choosealicense.git
# ! [rejected]        gh-pages -> gh-pages (fetch first)
# error: failed to push some refs to 'http://IP'
```

```
# hint: Updates were rejected because the remote contains work that
you do
# hint: not have locally. This is usually caused by another
repository pushing
# hint: to the same ref. You may want to first integrate the remote
changes
# hint: (e.g., 'git pull ...') before pushing again.
# hint: See the 'Note about fast-forwards' in 'git push --help' for
details.
```

Si vous voyez ! [rejected] gh-pages -> gh-pages (fetch first) vous devez faire un "merge" de la branche distante "origin/gh-pages" avec la votre "gh-pages". On utilise git pull

```
git pull
# Username for 'http://IP': username
# Password for 'http://username@IP':
# remote: Counting objects: 4, done.
# remote: Compressing objects: 100% (2/2), done.
# remote: Total 3 (delta 1), reused 1 (delta 0)
# Unpacking objects: 100% (3/3), done.
# From http://IP/root/choosealicense.git
#   93848fa..13d14c9  gh-pages    -> origin/gh-pages
# Merge made by the 'recursive' strategy.
#  _licenses/john-doe-1.0.txt | 0
#  1 file changed, 0 insertions(+), 0 deletions(-)
#   create mode 100644 _licenses/john-doe-1.0.txt
```

Puis vous pouvez faire maintenant le git push

```
git push
# Username for 'http://IP': username
# Password for 'http://username@IP':
# Counting objects: 4, done.
# Delta compression using up to 8 threads.
# Compressing objects: 100% (4/4), done.
# Writing objects: 100% (4/4), 518 bytes | 0 bytes/s, done.
# Total 4 (delta 2), reused 0 (delta 0)
# To http://IP/root/choosealicense.git
#   13d14c9..819a887  gh-pages -> gh-pages
```

Regardez dans le dossier "\_licenses"

- Quels fichiers viennent des autres étudiants du TP ?
- Comment savoir qui a ajouté une license en particulier ?
- Regardez l'historique git, à quoi ressemble le graphe de branche ?
- Comment repérer les commits des autres étudiants du TP ?

Vous avez donc tous en local sur votre branche "gh-pages", le contenu ajouté des autres étudiants.

## La commande git fetch

La commande git fetch va récupérer les commits qui sont sur le serveur, mais ne va pas les appliquer comme git pull. Si personne n'a encore poussé de nouveaux commit, la commande ne retourne rien

```
git fetch
# Username for 'http://IP': username
# Password for 'http://username@IP':
```

S'il y a de nouveaux commit, la commande va afficher ce que git a récupéré, mais ne va rien appliquer.

```
git fetch
# Username for 'http://IP': username
# Password for 'http://username@IP':
# remote: Counting objects: 11, done.
# remote: Compressing objects: 100% (7/7), done.
# remote: Total 7 (delta 5), reused 0 (delta 0)
# Unpacking objects: 100% (7/7), done.
# From http://IP/root/choosealicense.git
# + abf1ba8...4c93465 gh-pages -> origin/gh-pages (forced
update)
```

Règle générale, vous pouvez toujours utiliser git pull

## Exercice 5 : résoudre les conflits

Modifier le fichier "spec/license\_meta\_spec.rb", dans le tableau "legacy", ajouter votre license à la première ligne.

```
...
  legacy = [
    'alexandre-dubreuil-1.0.txt', # Ajouter votre ligne ici
    'afl-3.0',
    'agpl-3.0',
    'artistic-2.0',
    'bsd-2-clause',
    'bsd-3-clause',
  ]
...
```

Commitez le fichier et faites un git push. Si vous avez besoin de faire un git pull, vous allez entrer en conflit, puisque tout le monde a modifié la même ligne.

```

git pull
# Username for 'http://IP': username
# Password for 'http://username@IP':
# remote: Counting objects: 7, done.
# remote: Compressing objects: 100% (4/4), done.
# remote: Total 4 (delta 3), reused 0 (delta 0)
# Unpacking objects: 100% (4/4), done.
# From http://IP/root/chooseallicence.git
#   93848fa..abf1ba8  gh-pages      -> origin/gh-pages
# Auto-merging spec/license_meta_spec.rb
# CONFLICT (content): Merge conflict in spec/license_meta_spec.rb
# Automatic merge failed; fix conflicts and then commit the result.

git status
# On branch gh-pages
# Your branch and 'origin/gh-pages' have diverged,
# and have 2 and 1 different commits each, respectively.
#   (use "git pull" to merge the remote branch into yours)
# You have unmerged paths.
#   (fix conflicts and run "git commit")
#   (use "git merge --abort" to abort the merge)
#
# Unmerged paths:
#   (use "git add <file>..." to mark resolution)
#
#       both modified:   spec/license_meta_spec.rb
#
# no changes added to commit (use "git add" and/or "git commit -a")

```

Vous êtes en conflit sur le fichier "both modified: spec/license\_meta\_spec.rb". Vous devez l'ouvrir, garder les deux lignes du conflit, commiter, puis faire un git push.

Autrement dit, les lignes suivantes :

```

<<<<<<< HEAD
      'john-doe-1.0.txt',
=====
      'alexandre-dubreuil-1.0.txt',
>>>>>>> abf1ba89021c1ac0571fc04429df5f9fb476dc45

```

Deviennent :

```

      'john-doe-1.0.txt',
      'alexandre-dubreuil-1.0.txt',

```

On garde les 2 lignes parce qu'on souhaite garder la modification des 2 personnes. Chaque résolution de conflit est différente.

**Exercice 6 : créer une branche distante**

Vous pouvez aussi créer des branches sur le serveur git. Pour cela il faut pousser (git push) la branche sur le serveur.

Par exemple, pour créer une branche qui s'appelle "alexandre-dubreuil" sur le serveur, et qui contient le même contenu que gh-pages, vous pouvez faire la commande suivante. La notation gh-pages:alexandre-dubreuil veut dire la branche gh-pages en local va servir de référence pour créer la branche alexandre-dubreuil sur le serveur.

```
git push origin gh-pages:alexandre-dubreuil
# Username for 'http://IP': username
# Password for 'http://username@IP':
# Counting objects: 4, done.
# Delta compression using up to 8 threads.
# Compressing objects: 100% (4/4), done.
# Writing objects: 100% (4/4), 411 bytes | 0 bytes/s, done.
# Total 4 (delta 3), reused 0 (delta 0)
# To http://IP/root/chooseallicence.git
# * [new branch]      gh-pages -> alexandre-dubreuil
```

Pour la supprimer, on utilise aussi git push

```
git push --delete origin alexandre-dubreuil
# Username for 'http://IP': username
# Password for 'http://username@IP':
# To http://IP/root/chooseallicence.git
# - [deleted]          alexandre-dubreuil
```

**Exercice 7 : SVN**

Subversion (git) est un gestionnaire de source centralisé. Il propose le même usage que git, mais ne permet pas de travailler en mode "offline", c'est-à-dire faire des commits sans faire de "push".

```
# Équivalent de "git init"
mkdir svn-repository
cd svn-repository
svnadmin create tp-svn
svn import /home/user/svn-repository/tp-svn file:///home/user/svn-repository/tp-svn/trunk -m "Initial import of project1"
# Il faut faire un checkout dans un autre dossier que le precedent
/home/user/svn-repository/tp-svn
svn co file:///home/user/svn-repository/tp-svn/trunk /home/user/tp-svn
touch file
```

```
# Équivalent de "git add", "git rm", "git mv"
svn add file
svn rm file
svn mv file
# Équivalent de "git status", "git diff"
svn status
svn diff
```

La commande commit est différente pour svn : la commande commit va immédiatement envoyer la version vers le dépôt distant (svn est toujours synchronisé avec le distant). Si le dépôt est indisponible, vous ne pouvez pas commiter

```
# Équivalent de "git commit" + "git push"
svn commit
# Équivalent de "git log"
svn log
# Équivalent de "git pull"
svn update
```

Pour faire des tags et des branches dans svn, il faut copier le dépôt. Par défaut la branche principale dans svn s'appelle "trunk" (équivalent de "master")

```
# Équivalent de "git tag name"
svn copy file:///home/user/svn-repository/tp-svn/trunk
file:///home/user/svn-repository/tp-svn/tags/name

# Équivalent de "git branch name"
svn copy file:///home/user/svn-repository/tp-svn/trunk
file:///home/user/svn-repository/tp-svn/branches/name

# Équivalent de "git checkout name" pour changer de branche
svn switch file:///home/user/svn-repository/tp-svn/branches/names

# Équivalent de "git branch"
svn list file:///home/user/svn-repository/tp-svn/branches
```

## Bonus : créer son dépôt git

### Option 1 : utiliser un service existant

- <https://github.com> : plus connu, mais pas de dépôts privés gratuits
- <https://about.gitlab.com/> : permet d'avoir des dépôts privés gratuitement (utile pour vos projets)



## Option 2 : sur son propre serveur

- Installer ssh et git sur le serveur
- Créer un nouveau dépôt git sur le serveur avec `git init depot`
- Éditer le fichier `.git/config`, ajouter la ligne `bare = true`
- Sur une autre machine, faire `git clone utilisateur@ip:depot`, en modifiant l'utilisateur, l'ip et le chemin vers le depot